

23-1 PROCESS-TO-PROCESS DELIVERY

The transport layer is responsible for process-to-process delivery—the delivery of a packet, part of a message, from one process to another. Two processes communicate in a client/server relationship, as we will see later.

Topics discussed in this section:

Client/Server Paradigm

Multiplexing and Demultiplexing

Connectionless Versus Connection-Oriented Service

Reliable Versus Unreliable

Three Protocols

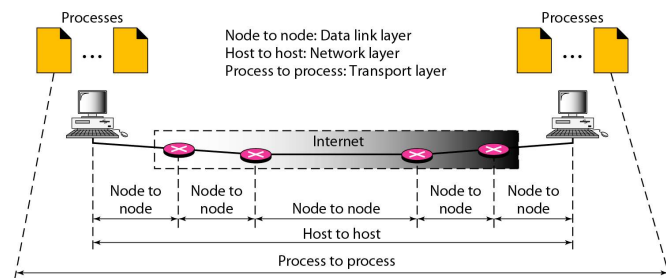
23.1

Note

The transport layer is responsible for process-to-process delivery.

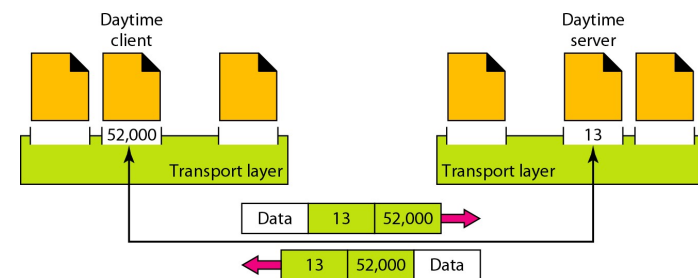
23.2

Figure 23.1 Types of data deliveries

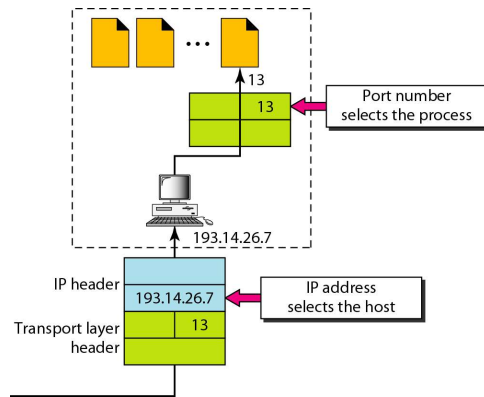


23.3

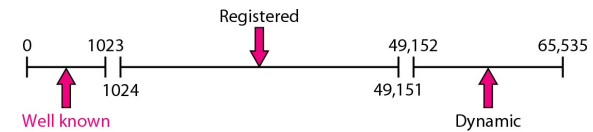
Figure 23.2 Port numbers



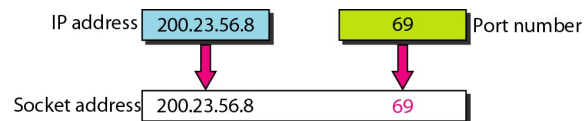
23.4

Figure 23.3 IP addresses versus port numbers

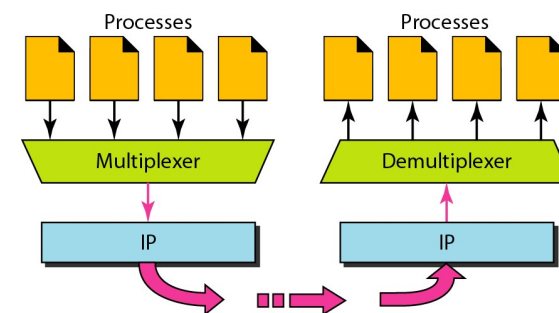
23.5

Figure 23.4 IANA ranges

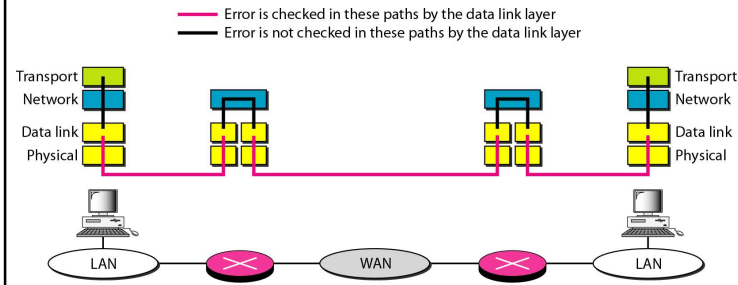
23.6

Figure 23.5 Socket address

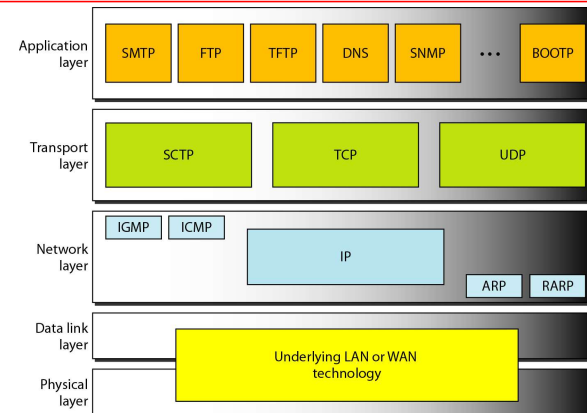
23.7

Figure 23.6 Multiplexing and demultiplexing

23.8

Figure 23.7 Error control

23.9

Figure 23.8 Position of UDP, TCP, and SCTP in TCP/IP suite

23.10

23-2 USERDATAGRAM PROTOCOL(UDP)

The User Datagram Protocol (UDP) is called a connectionless, unreliable transport protocol. It does not add anything to the services of IP except to provide process-to-process communication instead of host-to-host communication.

Topics discussed in this section:

Well-Known Ports for UDP

User Datagram

Checksum

UDP Operation

Use of UDP

23.11

Table 23.1 Well-known ports used with UDP

Port	Protocol	Description
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	BOOTPs	Server port to download bootstrap information
68	BOOTPc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

23.12

Example 23.1

In UNIX, the well-known ports are stored in a file called `/etc/services`. Each line in this file gives the name of the server and the well-known port number. We can use the `grep` utility to extract the line corresponding to the desired application. The following shows the port for FTP. Note that FTP can use port 21 with either UDP or TCP.

```
$ grep ftp /etc/services
ftp      21/tcp
ftp      21/udp
```

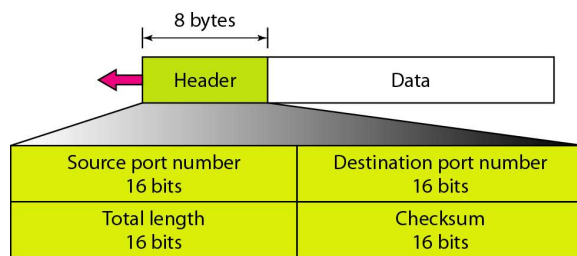
23.13

Example 23.1 (continued)

SNMP uses two port numbers (161 and 162), each for a different purpose, as we will see in Chapter 28.

```
$ grep snmp /etc/services
snmp      161/tcp      #Simple Net  Mgmt Proto
snmp      161/udp      #Simple Net  Mgmt Proto
snmptrap  162/udp      #Traps for SNMP
```

23.14

Figure 23.9 User datagram format

23.15

Note

**UDP length
= IP length – IP header's length**

23.16

23-3 TCP

TCP is a connection-oriented protocol; it creates a virtual connection between two TCPs to send data. In addition, TCP uses flow and error control mechanisms at the transport level.

Topics discussed in this section:

TCP Services

TCP Features

Segment

ATCP Connection

Flow Control

Error Control

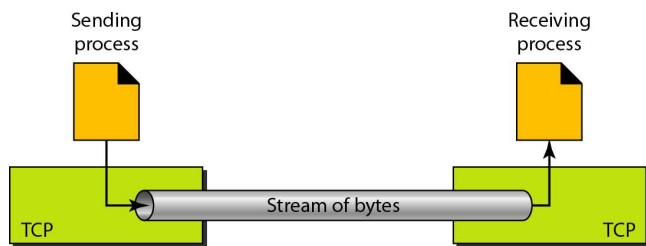
23.21

Table 23.2 Well-known ports used by TCP

Port	Protocol	Description
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

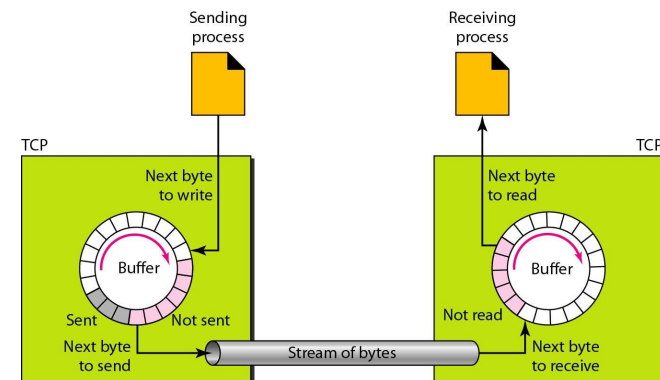
23.22

Figure 23.13 Stream delivery

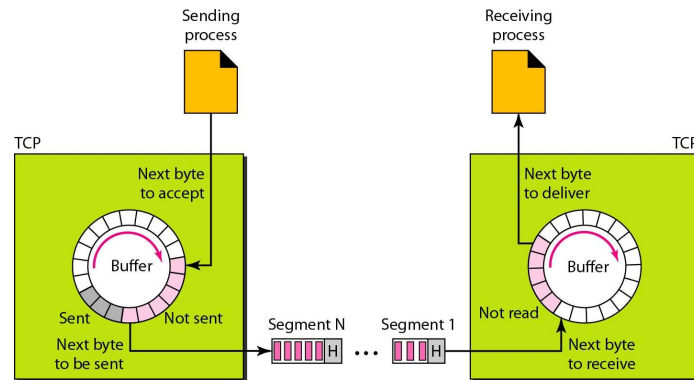


23.23

Figure 23.14 Sending and receiving buffers



23.24

Figure 23.15 TCP segments

23.25

Note

The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with a randomly generated number.

23.26

Example 23.3

The following shows the sequence number for each segment:

Segment 1	➡	Sequence Number: 10,001 (range: 10,001 to 11,000)
Segment 2	➡	Sequence Number: 11,001 (range: 11,001 to 12,000)
Segment 3	➡	Sequence Number: 12,001 (range: 12,001 to 13,000)
Segment 4	➡	Sequence Number: 13,001 (range: 13,001 to 14,000)
Segment 5	➡	Sequence Number: 14,001 (range: 14,001 to 15,000)

23.27

Note

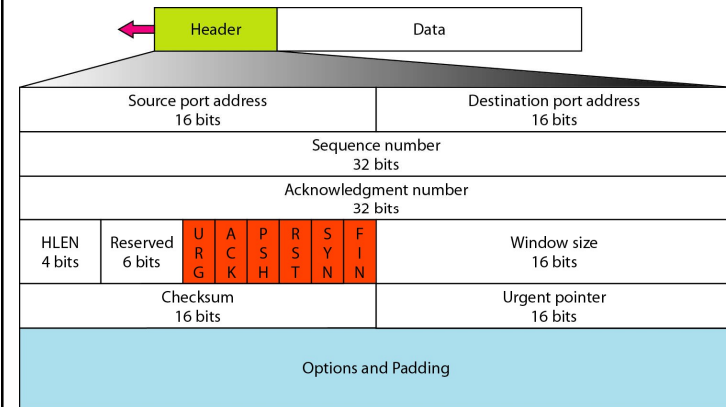
The value in the sequence number field of a segment defines the number of the first data byte contained in that segment.

23.28

Note

The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive. The acknowledgment number is cumulative.

23.29

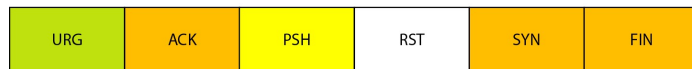
Figure 23.16 *TCP segment format*

23.30

Figure 23.17 *Control field*

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



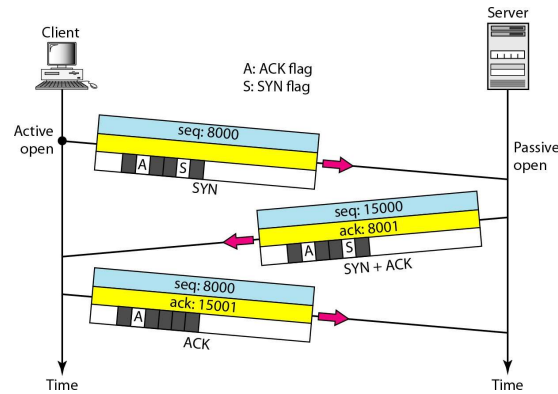
23.31

Table 23.3 *Description of flags in the control field*

Flag	Description
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

23.32

Figure 23.18 Connection establishment using three-way handshaking



23.33

Note

A SYN segment cannot carry data, but it consumes one sequence number.

23.34

Note

A SYN + ACK segment cannot carry data, but does consume one sequence number.

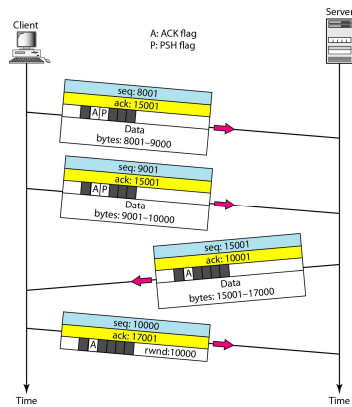
23.35

Note

An ACK segment, if carrying no data, consumes no sequence number.

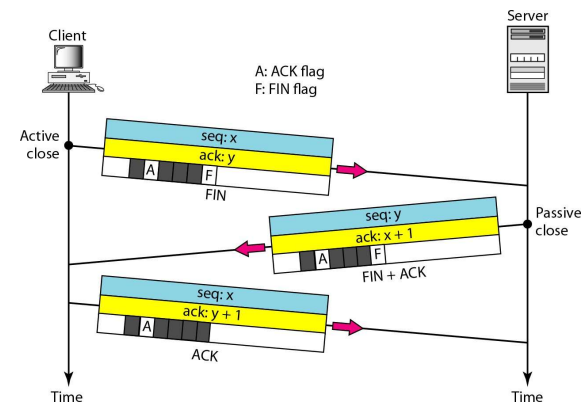
23.36

Figure 23.19 Data transfer



23.37

Figure 23.20 Connection termination using three-way handshaking



23.38

Note

The FIN segment consumes one sequence number if it does not carry data.

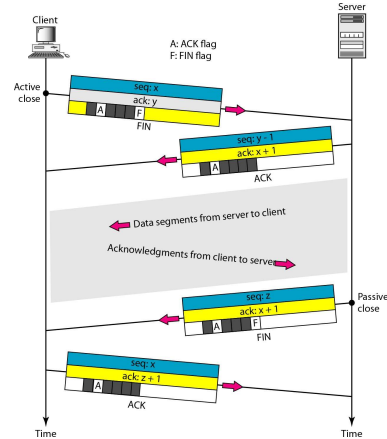
23.39

Note

The FIN + ACK segment consumes one sequence number if it does not carry data.

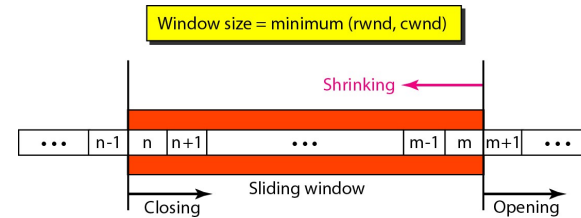
23.40

Figure 23.21 Half-close



23.41

Figure 23.22 Sliding window



23.42

Note

A sliding window is used to make transmission more efficient as well as to control the flow of data so that the destination does not become overwhelmed with data. TCP sliding windows are byte-oriented.

23.43

Example 23.4

What is the value of the receiver window (rwnd) for host A if the receiver, host B, has a buffer size of 5000 bytes and 1000 bytes of received and unprocessed data?

Solution

The value of $rwnd = 5000 - 1000 = 4000$. Host B can receive only 4000 bytes of data before overflowing its buffer. Host B advertises this value in its next segment to A.

23.44

Example 23.5

What is the size of the window for host A if the value of *rwnd* is 3000 bytes and the value of *cwnd* is 3500 bytes?

Solution

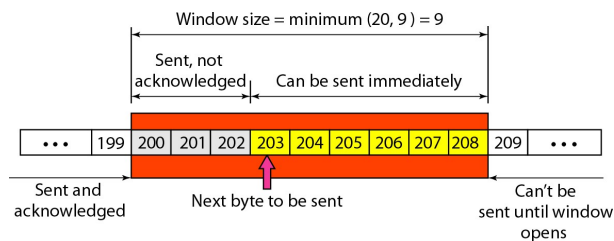
The size of the window is the smaller of *rwnd* and *cwnd*, which is 3000 bytes.

23.45

Example 23.6

Figure 23.23 shows an unrealistic example of a sliding window. The sender has sent bytes up to 202. We assume that *cwnd* is 20 (in reality this value is thousands of bytes). The receiver has sent an acknowledgment number of 200 with an *rwnd* of 9 bytes (in reality this value is thousands of bytes). The size of the sender window is the minimum of *rwnd* and *cwnd*, or 9 bytes. Bytes 200 to 202 are sent, but not acknowledged. Bytes 203 to 208 can be sent without worrying about acknowledgment. Bytes 209 and above cannot be sent.

23.46

Figure 23.23 Example 23.6

23.47

Note**Some points about TCP sliding windows:**

- ❑ The size of the window is the lesser of *rwnd* and *cwnd*.
- ❑ The source does not have to send a full window's worth of data.
- ❑ The window can be opened or closed by the receiver, but should not be shrunk.
- ❑ The destination can send an acknowledgment at any time as long as it does not result in a shrinking window.
- ❑ The receiver can temporarily shut down the window; the sender, however, can always send a segment of 1 byte after the window is shut down.

23.48

Note

ACK segments do not consume sequence numbers and are not acknowledged.

23.49

Note

In modern implementations, a retransmission occurs if the retransmission timer expires or three duplicate ACK segments have arrived.

23.50

Note

No retransmission timer is set for an ACK segment.

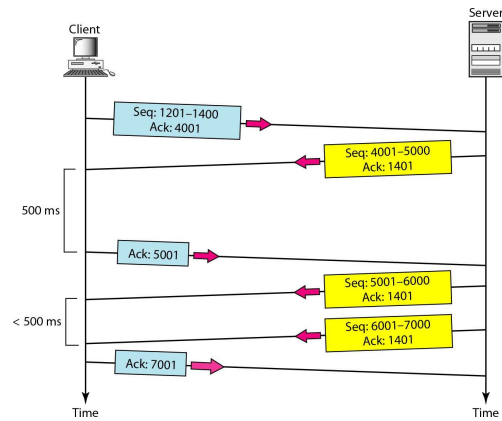
23.51

Note

Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order segment is delivered to the process.

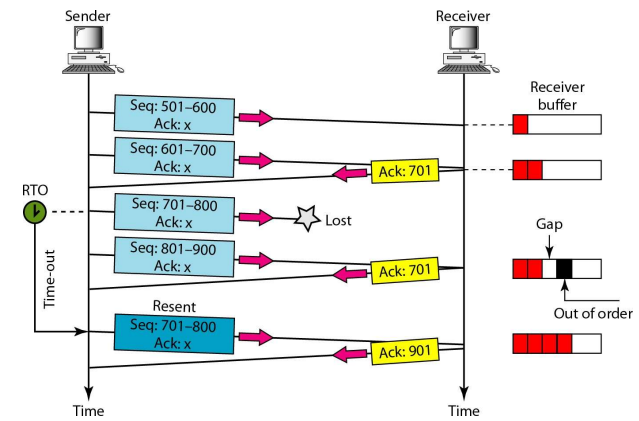
23.52

Figure 23.24 Normal operation



23.53

Figure 23.25 Lost segment



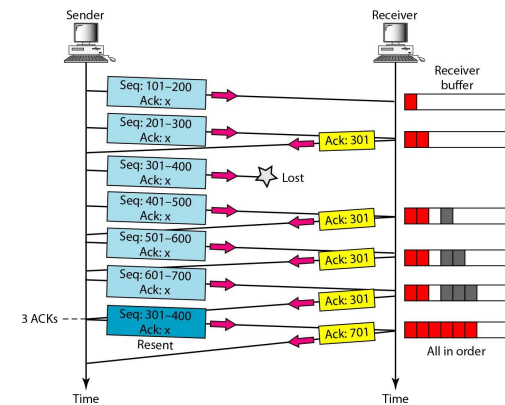
23.54

Note

The receiver TCP delivers only ordered data to the process.

23.55

Figure 23.26 Fast retransmission



23.56

24-1 DATATRAFFIC

The main focus of congestion control and quality of service is **data traffic**. In congestion control we try to avoid traffic congestion. In quality of service, we try to create an appropriate environment for the traffic. So, before talking about congestion control and quality of service, we discuss the data traffic itself.

Topics discussed in this section:

Traffic Descriptor
Traffic Profiles

24.2

24-2 CONGESTION

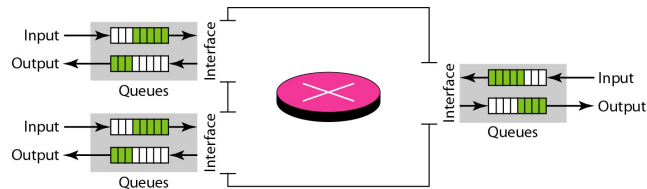
Congestion in a network may occur if the load on the network—the number of packets sent to the network—is greater than the capacity of the network—the number of packets a network can handle. Congestion control refers to the mechanisms and techniques to control the congestion and keep the load below the capacity.

Topics discussed in this section:

Network Performance

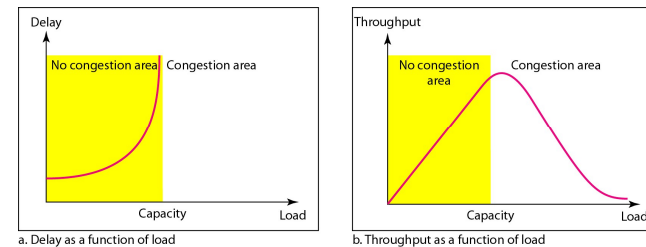
24.5

Figure 24.3 Queues in a router



24.6

Figure Packet delay and throughput as functions of load



24.7

24-3 CONGESTIONCONTROL

Congestion control refers to techniques and mechanisms that can either prevent congestion, before it happens, or remove congestion, after it has happened. In general, we can divide congestion control mechanisms into two broad categories: open-loop congestion control (prevention) and closed-loop congestion control (removal).

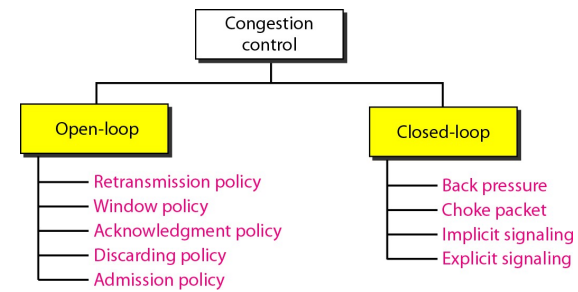
Topics discussed in this section:

Open-Loop Congestion Control

Closed-Loop Congestion Control

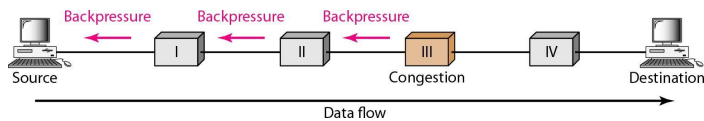
24.8

Figure 24.5 Congestion control categories



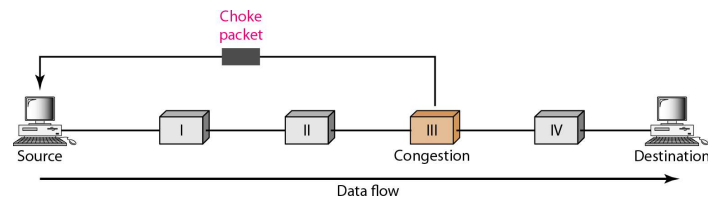
24.9

Figure 24.6 Backpressure method for alleviating congestion



24.10

Figure 24.7 Choke packet



24.11

24-4 TWOEXAMPLES

To better understand the concept of congestion control, let us give two examples: one in TCP and the other in Frame Relay.

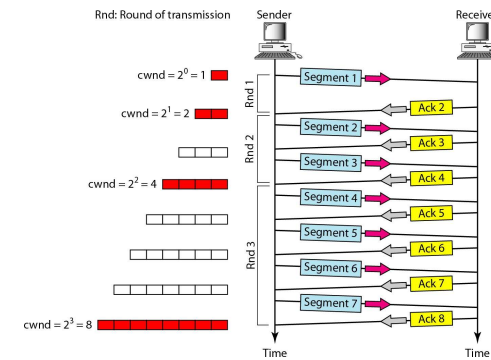
Topics discussed in this section:

Congestion Control in TCP

Congestion Control in Frame Relay

24.12

Figure 24.8 *Slow start, exponential increase*



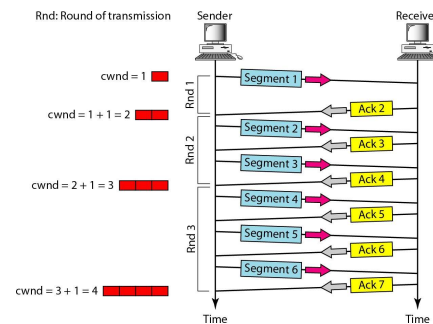
24.13

Note

In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

24.14

Figure 24.9 *Congestion avoidance, additive increase*



24.15

Note

In the congestion avoidance algorithm, the size of the congestion window increases additively until congestion is detected.

24.16

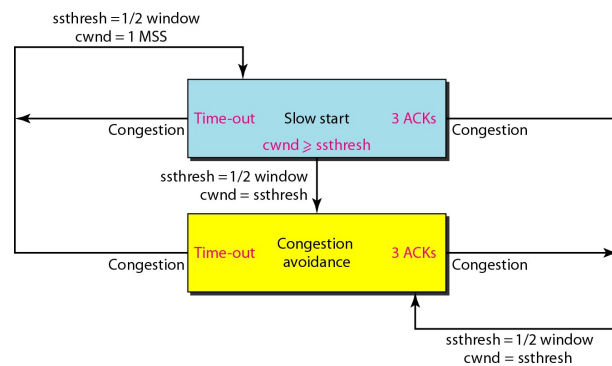
Note

An implementation reacts to congestion detection in one of the following ways:

- ❑ If detection is by time-out, a new slow start phase starts.
- ❑ If detection is by three ACKs, a new congestion avoidance phase starts.

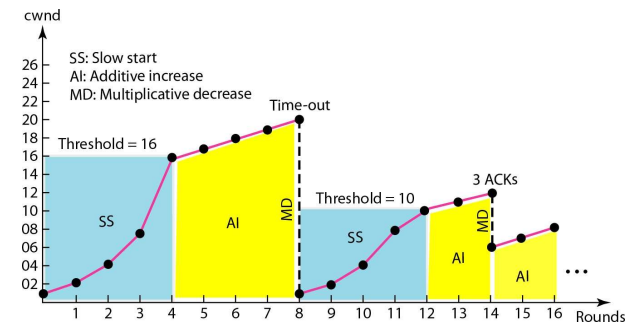
24.17

Figure 24.10 TCP congestion policy summary

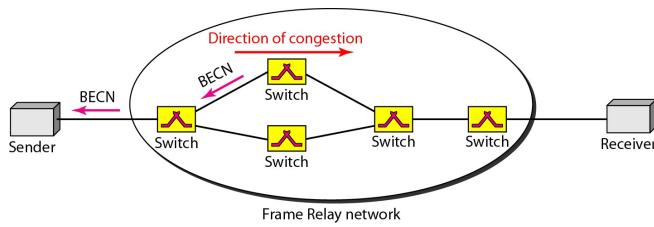


24.18

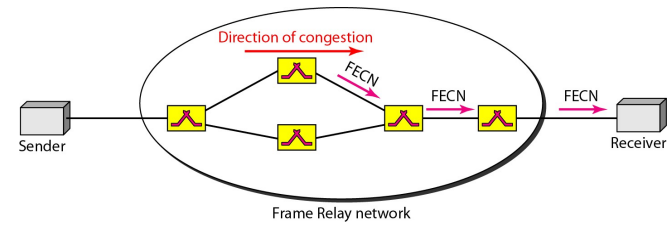
Figure 24.1 Congestion example



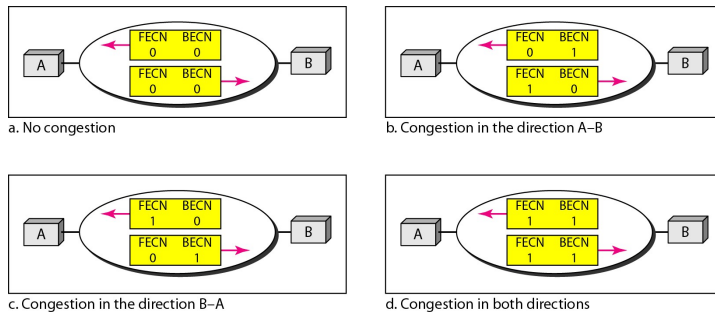
24.19

Figure 24.12 *BECN*

24.20

Figure 24.13 *FECN*

24.21

Figure 24.14 *Four cases of congestion*

24.22